CO1 – Assessment Tool 3 — Questions and Solutions

Question 1: Matrix Operations and Analysis

Design an R program to:

- · Create two 3 × 3 matrices with user-defined values.
- · Perform addition, subtraction, and multiplication of the matrices.
- · Find the transpose of each matrix.
- · Calculate the row sums, column sums, and determinant of the first matrix.
- · Display the results with appropriate labels.

Aim

To perform basic matrix operations such as addition, subtraction, multiplication, transpose, and to calculate row sums, column sums and determinant of a matrix using R.

Program

```
# Question 1: Matrix Operations and Analysis

# Create two 3x3 matrices with user-defined values
matrix1 <- matrix(c(4, 8, 2,
                    6, 1, 9,
                    3, 7, 5), nrow = 3, byrow = TRUE)

matrix2 <- matrix(c(1, 2, 3,
                    4, 5, 6,
                    7, 8, 9), nrow = 3, byrow = TRUE)

cat("Matrix 1:\n")
print(matrix1)
cat("\nMatrix 2:\n")
print(matrix2)

# Addition of the two matrices
mat_sum <- matrix1 + matrix2
cat("\nAddition (Matrix1 + Matrix2):\n")
print(mat_sum)

# Subtraction of the two matrices
mat_diff <- matrix1 - matrix2
cat("\nSubtraction (Matrix1 - Matrix2):\n")
print(mat_diff)

# Matrix multiplication
mat_prod <- matrix1 %*% matrix2
cat("\nMultiplication (Matrix1 %*% Matrix2):\n")
print(mat_prod)

# Transpose of each matrix
cat("\nTranspose of Matrix 1:\n")
print(t(matrix1))
cat("\nTranspose of Matrix 2:\n")
print(t(matrix2))

# Row sums, column sums and determinant of Matrix 1
```

```
cat("\nRow sums of Matrix 1   :", rowSums(matrix1), "\n")
cat("Column sums of Matrix 1 :", colSums(matrix1), "\n")
cat("Determinant of Matrix 1 :", det(matrix1), "\n")
```

Sample Output

```
Matrix 1:
     [,1] [,2] [,3]
[1,]    4    8    2
[2,]    6    1    9
[3,]    3    7    5

Addition (Matrix1 + Matrix2):
     [,1] [,2] [,3]
[1,]    5   10    5
[2,]   10    6   15
[3,]   10   15   14

Row sums of Matrix 1   : 14 16 15
Column sums of Matrix 1 : 13 16 16
Determinant of Matrix 1 : -280
```

Result

The program successfully performed addition, subtraction, multiplication, transpose, row/column sums and determinant of the matrices with proper labels.

Question 2: Array Operations and Data Analysis

Develop an R program to:

· Create a 3-dimensional array representing the marks of students in three subjects across two semesters.

· Calculate the average marks for each student.

· Find the highest and lowest marks in the array.

· Display the marks semester-wise and subject-wise using appropriate indexing.

Aim

To create and analyze a 3-dimensional array of student marks across subjects and semesters using R.

Program

```
# Question 2: Array Operations and Data Analysis

students  <- c("S1", "S2", "S3", "S4")
subjects  <- c("Maths", "Physics", "Chemistry")
semesters <- c("Sem1", "Sem2")

# Create a 3D array: students x subjects x semesters
marks_array <- array(
  c(78, 85, 90, 65,   72, 88, 95, 60,   80, 91, 70, 55,
    60, 77, 82, 68,   75, 89, 92, 58,   66, 79, 85, 71),
  dim = c(4, 3, 2),
  dimnames = list(students, subjects, semesters)
)

cat("3D Marks Array:\n")
print(marks_array)

# Average marks for each student (across all subjects & semesters)
avg_marks <- apply(marks_array, 1, mean)
cat("\nAverage marks for each student:\n")
print(round(avg_marks, 2))

# Highest and lowest marks in the array
cat("\nHighest mark in the array:", max(marks_array), "\n")
cat("Lowest mark in the array :", min(marks_array), "\n")

# Marks displayed semester-wise
cat("\nMarks in Semester 1:\n")
print(marks_array[, , "Sem1"])
cat("\nMarks in Semester 2:\n")
print(marks_array[, , "Sem2"])

# Marks displayed subject-wise (e.g., Maths, all students & semesters)
cat("\nMaths marks (all students, both semesters):\n")
print(marks_array[, "Maths", ])
```

Sample Output

```
Average marks for each student:
   S1    S2    S3    S4
71.83 84.83 85.67 62.83

Highest mark in the array: 95
```

```
Lowest mark in the array : 55

Marks in Semester 1:
    Maths Physics Chemistry
S1     78      72        80
S2     85      88        91
S3     90      95        70
S4     65      60        55
```

Result

The program successfully created a 3D array and computed the average, highest, lowest, and semester/subject-wise marks.

Question 3: R Calculations Using Functions

Write an R program to:

· Accept a numeric vector containing 20 integers.

· Calculate the sum, mean, median, variance, standard deviation, maximum, minimum, and range.

· Display all calculated statistics in a well-formatted output.

· Write a user-defined function to identify whether the mean is greater than the median.

Aim

To calculate descriptive statistics of a numeric vector and compare the mean with the median using a user-defined function.

Program

```
# Question 3: R Calculations Using Functions

# A numeric vector containing 20 integers
marks_vector <- c(45, 67, 89, 23, 56, 78, 90, 34, 65, 87,
                  12, 98, 76, 54, 43, 32, 61, 71, 81, 91)

# Calculate the required statistics
total_sum    <- sum(marks_vector)
mean_val     <- mean(marks_vector)
median_val   <- median(marks_vector)
variance_val <- var(marks_vector)
sd_val       <- sd(marks_vector)
max_val      <- max(marks_vector)
min_val      <- min(marks_vector)
range_val    <- range(marks_vector)

# Display all statistics in a well-formatted output
cat("---- Statistics of the Vector ----\n")
cat("Sum       :", total_sum, "\n")
cat("Mean      :", round(mean_val, 2), "\n")
cat("Median    :", median_val, "\n")
cat("Variance  :", round(variance_val, 2), "\n")
cat("Std Dev   :", round(sd_val, 2), "\n")
cat("Maximum   :", max_val, "\n")
cat("Minimum   :", min_val, "\n")
cat("Range     :", range_val[1], "to", range_val[2], "\n")

# User-defined function to compare mean and median
compare_mean_median <- function(m, med) {
  if (m > med) {
    return("Mean is greater than Median")
  } else if (m < med) {
    return("Mean is less than Median")
  } else {
    return("Mean is equal to Median")
  }
}

cat("\nComparison Result:", compare_mean_median(mean_val, median_val), "\n")
```

Sample Output

```
---- Statistics of the Vector ----
```

```
Sum       : 1253
Mean      : 62.65
Median    : 66
Variance  : 613.19
Std Dev   : 24.76
Maximum   : 98
Minimum   : 12
Range     : 12 to 98

Comparison Result: Mean is less than Median
```

Result

The program correctly computed all required statistics and determined that the mean is less than the median.

Question 4: Student Management System Using S3 Class

Develop an S3 class named Student with attributes: Student ID, Name, Department, Marks. Implement methods to:

- · Display the student details.
- · Calculate the average marks.
- · Assign grades based on marks.
- · Print the student record in a user-friendly format.

Aim

To implement a Student Management System using R's S3 class with methods for displaying details, computing average marks, and assigning a grade.

Program

```
# Question 4: Student Management System Using S3 Class

# Constructor function for the S3 class "Student"
Student <- function(id, name, department, marks) {
  student <- list(
    StudentID  = id,
    Name       = name,
    Department = department,
    Marks      = marks          # numeric vector of subject marks
  )
  class(student) <- "Student"
  return(student)
}

# Method to display student details
displayDetails <- function(student) UseMethod("displayDetails")
displayDetails.Student <- function(student) {
  cat("Student ID :", student$StudentID, "\n")
  cat("Name       :", student$Name, "\n")
  cat("Department :", student$Department, "\n")
  cat("Marks      :", student$Marks, "\n")
}

# Method to calculate average marks
averageMarks <- function(student) UseMethod("averageMarks")
averageMarks.Student <- function(student) {
  mean(student$Marks)
}

# Method to assign grade based on average marks
assignGrade <- function(student) UseMethod("assignGrade")
assignGrade.Student <- function(student) {
  avg <- averageMarks(student)
  if (avg >= 90) "A+"
  else if (avg >= 75) "A"
  else if (avg >= 60) "B"
  else if (avg >= 40) "C"
  else "Fail"
}

# S3 print method - prints record in a user-friendly format
print.Student <- function(x, ...) {
  cat("===== Student Record =====\n")
```

```
  displayDetails(x)
  cat("Average Marks :", round(averageMarks(x), 2), "\n")
  cat("Grade         :", assignGrade(x), "\n")
  cat("============================\n")
}

# Create a Student object and print the record
s1 <- Student(101, "Arun Kumar", "Computer Science", c(88, 92, 79, 85))
print(s1)
```

Sample Output

```
===== Student Record =====
Student ID : 101
Name       : Arun Kumar
Department : Computer Science
Marks      : 88 92 79 85
Average Marks : 86
Grade         : A
==========================
```

Result

The S3 class successfully stored the student data and displayed the details, average marks, and grade.

Question 5: Employee Management Using S4 and Reference Classes

Design an R application that:

· Creates an S4 class named Employee with slots for Employee ID, Name, Department, and Salary.

· Validates that the salary is a positive value.

· Creates a Reference Class named Payroll to update employee salaries and calculate annual salary.

· Demonstrates the difference between S4 objects and Reference Class objects by modifying employee details and displaying the updated results.

Aim

To manage employee records using an S4 class and a Reference Class, and to demonstrate the difference between their copy and reference semantics.

Program

```
# Question 5: Employee Management Using S4 and Reference Classes

# ---- S4 Class: Employee ----
setClass("Employee", representation(
  EmployeeID = "numeric",
  Name       = "character",
  Department = "character",
  Salary     = "numeric"
))

# Validity check: salary must be a positive value
setValidity("Employee", function(object) {
  if (object@Salary <= 0) "Salary must be a positive value"
  else TRUE
})

# S4 method to display employee details
setGeneric("showEmployee", function(emp) standardGeneric("showEmployee"))
setMethod("showEmployee", "Employee", function(emp) {
  cat("Employee ID :", emp@EmployeeID, "\n")
  cat("Name        :", emp@Name, "\n")
  cat("Department  :", emp@Department, "\n")
  cat("Salary      :", emp@Salary, "\n")
})

# Create an S4 Employee object
emp1 <- new("Employee", EmployeeID = 1, Name = "Priya Sharma",
            Department = "HR", Salary = 45000)

cat("---- Original S4 Employee ----\n")
showEmployee(emp1)

# S4 objects follow copy-on-modify semantics
emp2 <- emp1
emp2@Salary <- 50000

cat("\n---- After 'modifying' emp2 (S4 copy semantics) ----\n")
cat("emp1 Salary (unchanged):", emp1@Salary, "\n")
cat("emp2 Salary (changed)  :", emp2@Salary, "\n")

# ---- Reference Class: Payroll ----
Payroll <- setRefClass("Payroll",
  fields = list(
```

```
    EmployeeID = "numeric",
    Name       = "character",
    Salary     = "numeric"
  ),
  methods = list(
    updateSalary = function(newSalary) {
      Salary <<- newSalary
    },
    annualSalary = function() {
      return(Salary * 12)
    }
  )
)

# Create a Reference Class object
pay1 <- Payroll$new(EmployeeID = 1, Name = "Priya Sharma", Salary = 45000)
pay2 <- pay1        # pay2 refers to the SAME object as pay1

cat("\n---- Original Reference Class Payroll ----\n")
cat("pay1 Salary:", pay1$Salary, "\n")

# Update salary through pay2
pay2$updateSalary(50000)

cat("\n---- After updating pay2 (Reference Class semantics) ----\n")
cat("pay1 Salary (also changed):", pay1$Salary, "\n")
cat("pay2 Salary (changed)     :", pay2$Salary, "\n")
cat("Annual Salary (pay1)      :", pay1$annualSalary(), "\n")
```

Sample Output

```
---- Original S4 Employee ----
Employee ID : 1
Name        : Priya Sharma
Salary      : 45000

---- After 'modifying' emp2 (S4 copy semantics) ----
emp1 Salary (unchanged): 45000
emp2 Salary (changed)  : 50000

---- After updating pay2 (Reference Class semantics) ----
pay1 Salary (also changed): 50000
pay2 Salary (changed)     : 50000
Annual Salary (pay1)      : 600000
```

Result

The program demonstrated that modifying an S4 copy leaves the original object unchanged, while modifying a Reference Class object updates all references to it.